User guide

How to load documents, ask questions in everyday words, and check how well the system works. Click any screenshot to open it full size.

Getting started

This app answers questions from your own documents. You ask “Who appoints the Governor?” in plain language; it finds the matching passages even when the document uses different words, then writes a short answer with numbered citations. If the documents do not contain the answer, it says so instead of guessing.

The six steps

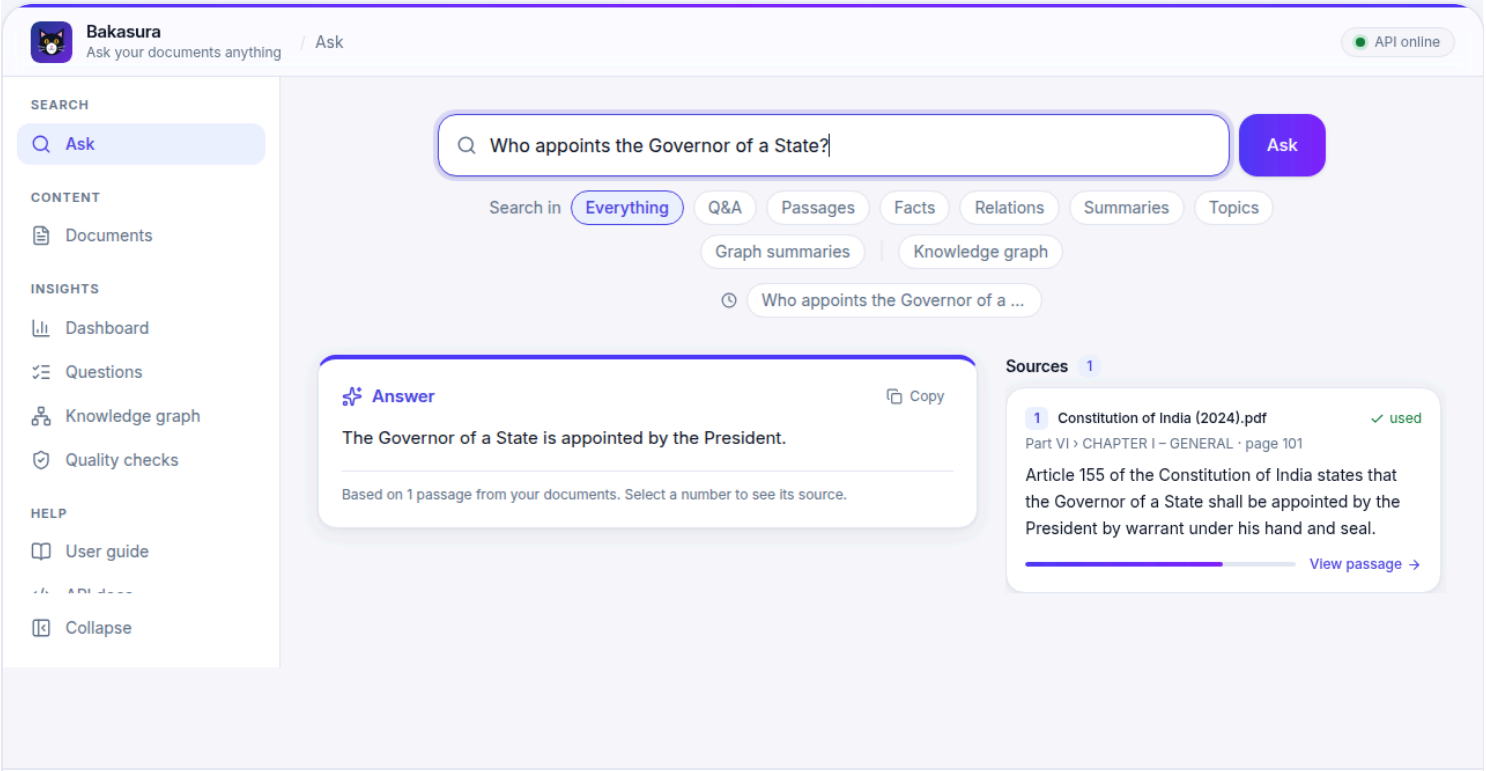
1. **Start the app** with `./start.sh` (Windows: `start.ps1`). It opens the API on port 8000 and this page on port 5173.
2. **Add documents** on the Documents page (PDF, Word or text).
3. **Ask** a question on the Ask page.
4. **Check the answer** by opening the cited passage.
5. **Measure** retrieval on the Dashboard, using the labelled Questions.
6. **Check the LLM’s work** on Quality checks.

The coloured dot at the top right shows whether the API is online. If it says “API offline”, start the app again.

What happens to a document

- It is cut into **sections** (about half a page, following the document’s headings) and each section into small **chunks** of about 65 words.
- An LLM rewrites vague words in each chunk (“It” becomes “The President”) and writes **Q&A pairs**, **factoids** and **relations** (who does what to whom).
- Everything is turned into vectors and stored. A search matches a small chunk, then the app reads its whole section, so answers have context.

Ask



The Ask page with a question, a written answer with citations, and source cards

1. Type a question in everyday words, or click one of the suggested questions. Press Enter.
2. The **answer** appears on the left. Numbered chips like **1** are citations: select one to highlight its source card.
3. The **Sources** panel on the right has a card for each passage that helped, rewritten as a short readable answer, with its document, section and page. A bar shows how close the passage is to your question. Cards marked **used** were cited in the answer; other helpful passages can appear without the mark.
4. **View passage** opens the full original text of the section, with buttons to read the previous and next one.

Filters

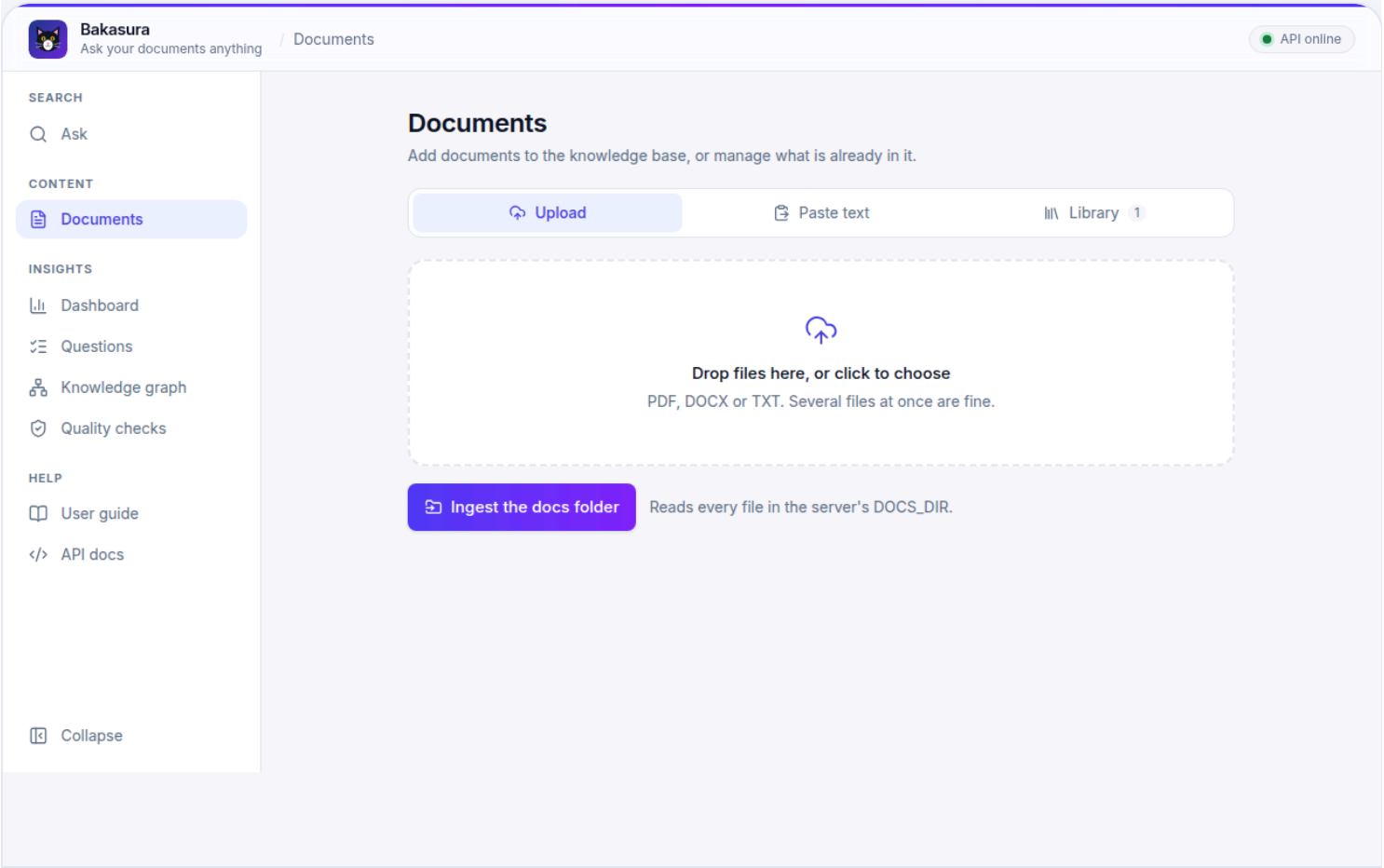
- **Search in** chips limit what is searched: Q&A, Passages, Facts, Relations, Summaries, Topics, Graph summaries. “Everything” (the default) searches the usual types (relations and graph summaries only when you pick them).
- **Knowledge graph** adds passages tied to the entities named in your question. On our test set it neither helped nor hurt, so it starts off. Passages it contributed are tagged “via graph”.

If the answer reads “I couldn’t find this in the documentation”, nothing relevant was retrieved. That is deliberate: the system abstains rather than invent an answer. Try different words, or add the missing

document.

The question is kept in the page address (`/?q=…`), so you can bookmark or share an answer.

Documents



The Documents page with the upload box and the three tabs

Add documents

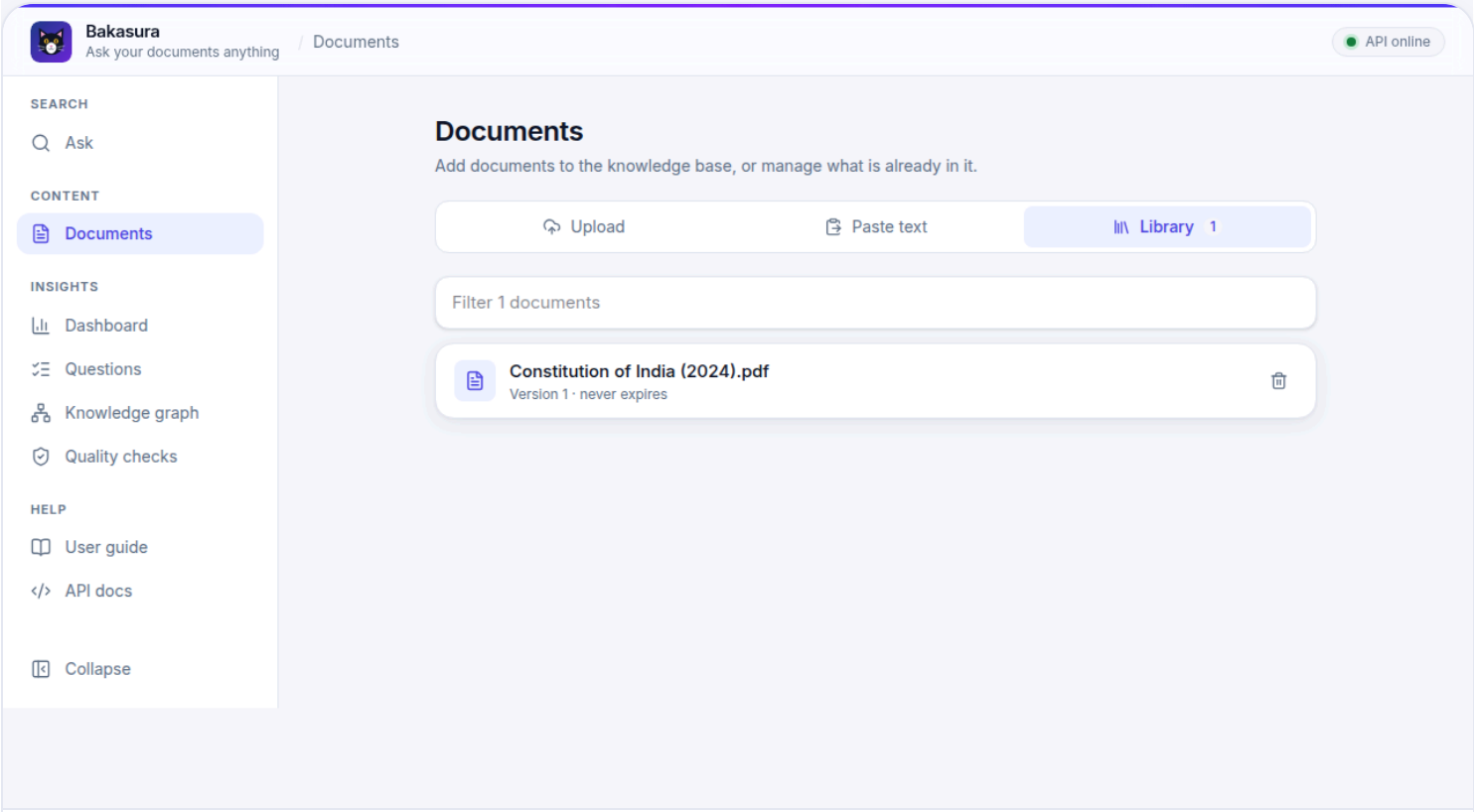
- **Upload**: drag files onto the box or click it. PDF, DOCX and TXT work. You can choose many files at once.
- **Ingest the docs folder** reads every file in the server's `DOCS_DIR` (`data/docs` by default).
- **Paste text**: give a title and paste the text. The title becomes the citation shown with answers.

Ingestion runs in the background and updates every second, so you can watch it work. For each file the table shows what it is doing now (reading the file, splitting, enriching chunks with the LLM, writing the summary and glossary, storing) and the counts grow as it goes: chunks, Q&A pairs, facts, triplets and terms. While chunks are being enriched the line under the progress bar says how many are done, for

example “120 of 2358 chunks”. Reading a large PDF shows no counts for a few minutes; that is the slow first step, not a hang.
How long the enrichment takes depends on the LLM provider you chose (see Running and configuring). Finished work is cached, so a repeated or resumed run is fast.

> The same text is never stored twice (it is recognised by a hash and shown as “skipped”).

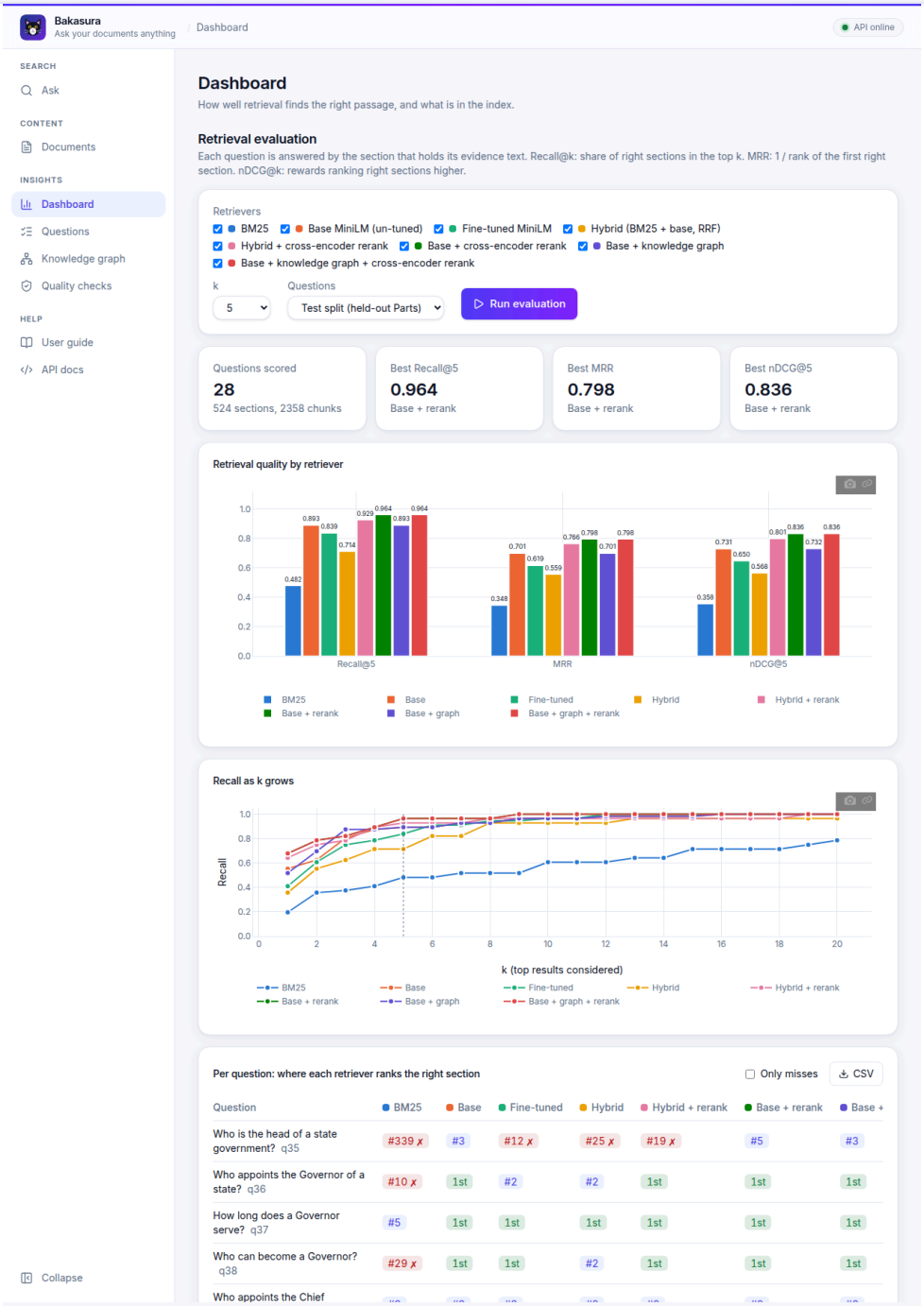
Library, versions and expiry



The Library tab listing documents with version and expiry

- The **Library** tab lists every document with its **version**, when it was stored and when it expires.
- **Updating**: add a file with the same name or title but changed text. The new version is built first, then the old one is deleted, so a failed update loses nothing. The version number goes up by one.
- **Expiry**: set `DOCUMENT_TTL_DAYS` and documents stored from then on expire after that many days. Expired documents are flushed every `FLUSH_INTERVAL_HOURS` (24 by default) or on demand with `POST /api/documents/flush`. With the default 0, nothing expires.
- **Delete** (the bin icon) removes a document and everything made from it. This cannot be undone.

Dashboard



The Dashboard comparing retrievers by Recall, MRR and nDCG

The Dashboard answers “which way of searching finds the right passage most often?” on the labelled questions.

Run an evaluation

1. Tick the retrievers, choose **k**, and choose the questions: the **test split** (held-out Parts, the honest number), train, or all.

2. Click **Run evaluation**. The first run embeds the whole index and takes a few minutes; later runs are quick. Progress is shown.
3. Read the comparison table and charts. Download the per-question table as CSV.

The retrievers

- **BM25**: keyword matching. The baseline.
- **Base MiniLM**: the stock embedding model. **Fine-tuned MiniLM**: the same model trained on this corpus (appears when configured).
- **Hybrid**: BM25 and the base model combined by reciprocal rank fusion.
- **+ cross-encoder rerank**: a second model re-orders the top 30 results. Ask uses base + rerank.
- **+ knowledge graph**: adds graph candidates before ranking.

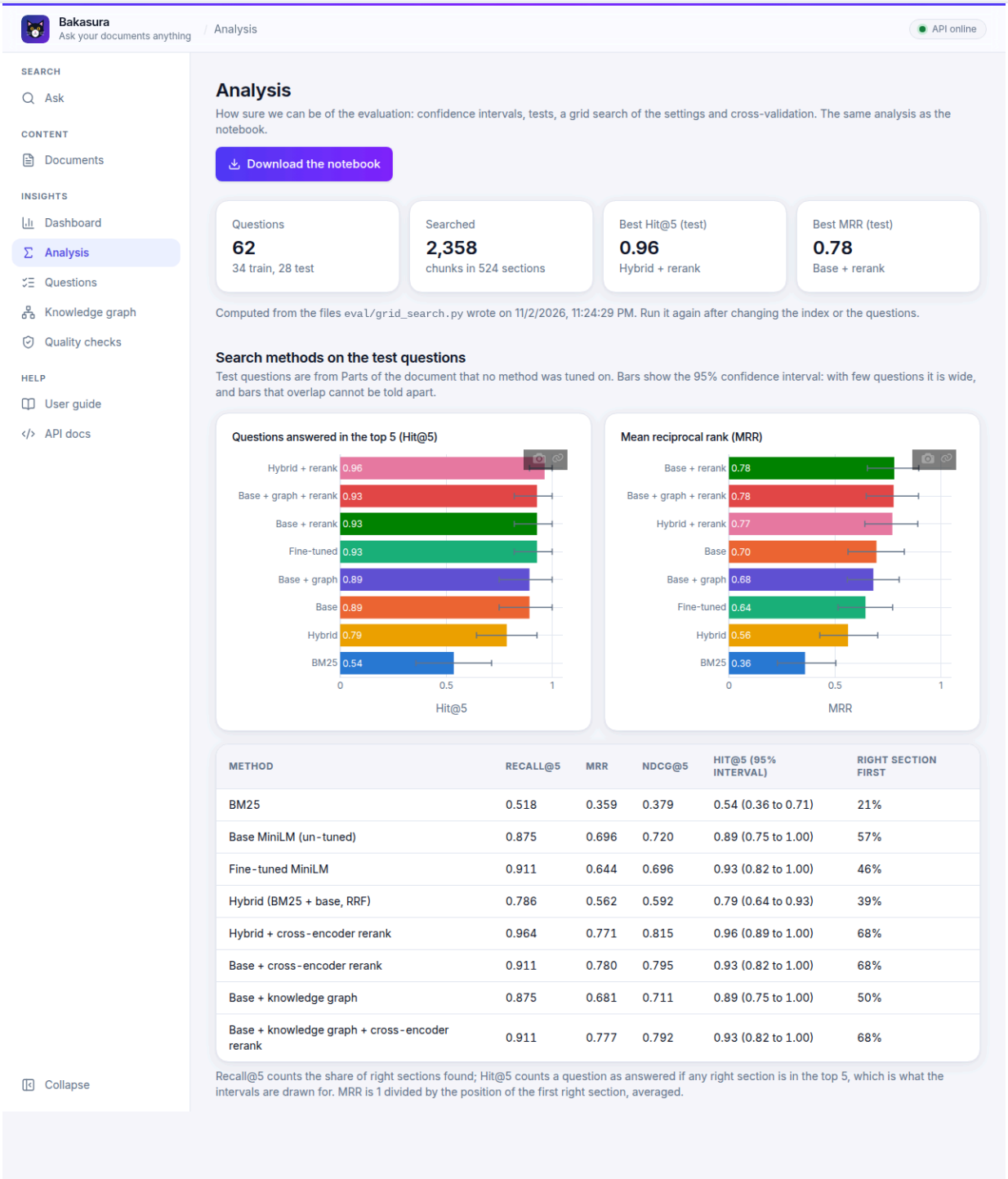
The metrics

- **Recall@k**: share of questions where the right section is in the top k.
- **MRR**: 1 divided by the rank of the first right section, averaged. Higher means the right answer sits nearer the top.
- **nDCG@k**: like MRR but rewards every right section and discounts lower ranks.

The per-question table shows the rank each retriever gave the right section; misses are highlighted. The index statistics show how many points of each type exist and how big the chunks are.

> With few test questions, a difference of one or two questions is noise. Treat small gaps as ties. The Analysis page shows which differences are real.

Analysis



The Analysis page with confidence-interval charts of each search method and a table of their scores

The Dashboard gives one number per search method. With a few dozen questions, one number can be luck, so this page shows **how sure we can be**: confidence intervals, tests of whether a difference is real, a grid search of the settings and cross-validation. It is the same analysis as the Jupyter notebook, built into the app. The **Download the notebook** button saves the notebook itself, with its charts and written findings.

How to read it

- **Search methods on the test questions.** Each bar is a score and the thin line on it is the 95% confidence interval: the range the true score probably lies in. **When two intervals overlap, the methods cannot be told apart** with these questions. *Hit@5* counts a question as answered if a right section is in the top 5; *Recall@5* counts the share of right sections found, so the two differ slightly for questions with several right sections. The last column is how often the right section is the very first result.
- **Are the differences real?** Two methods are compared on the same questions. real gap means the 95% interval of the difference excludes zero. not shown does **not** mean "no difference": it means these questions are too few to tell. The p-value is adjusted for making several comparisons, which makes a lucky result harder to claim. The fine-tuned model is compared on the test questions only, because it was trained on the others.
- **Grid search.** Three methods have settings nobody can know in advance: BM25's `k1` and `b`, the hybrid's fusion constant, and how many top results the reranker re-scores. Every setting was scored on every question. Flat lines mean the setting hardly matters.
- **Cross-validation by Part.** Choosing the best setting and reporting its score on the same questions is optimistic. Here the questions are split so that all questions of one Part stay together; the best setting is chosen on the other folds and scored on the held-out one. Compare the **gain** from tuning with the spread between folds in the last table: a gain smaller than that spread is easily luck.

Where the numbers come from

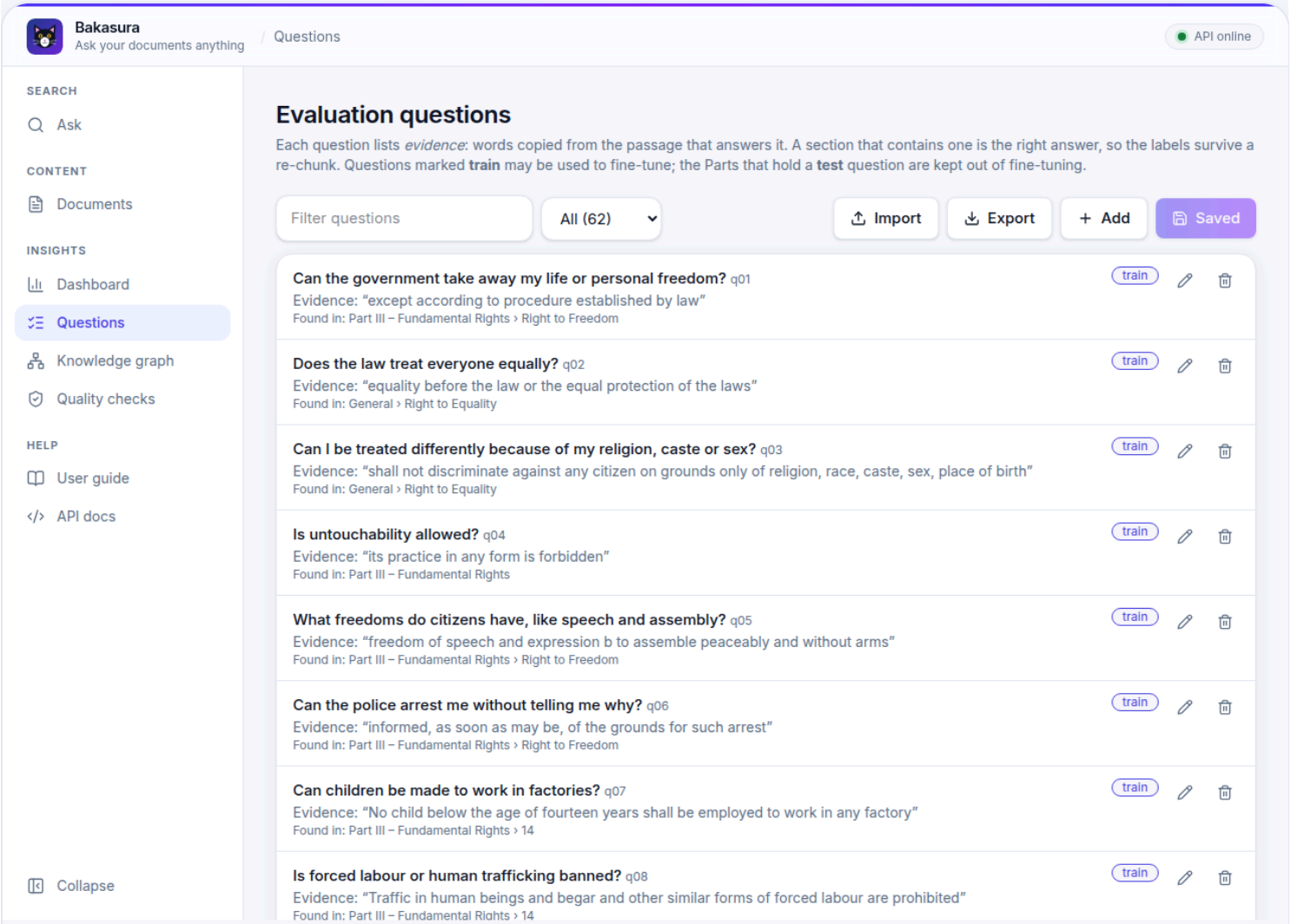
The page reads two files written by `eval/grid_search.py` (in `notebooks/data/`) and shows when they were written. Run it again after you re-ingest, or change the questions or the index; it needs Qdrant running with the documents ingested and takes about four minutes:

```
uv run --group eval --group notebook python eval/grid_search.py
```

To open the notebook itself: `uv run --group notebook jupyter lab notebooks/capstone_analysis.ipynb`. The page and the notebook give the same numbers; the intervals can differ in the third decimal because resampling is random.

> If the page says "No analysis data yet", run the command above once and reload.

Questions



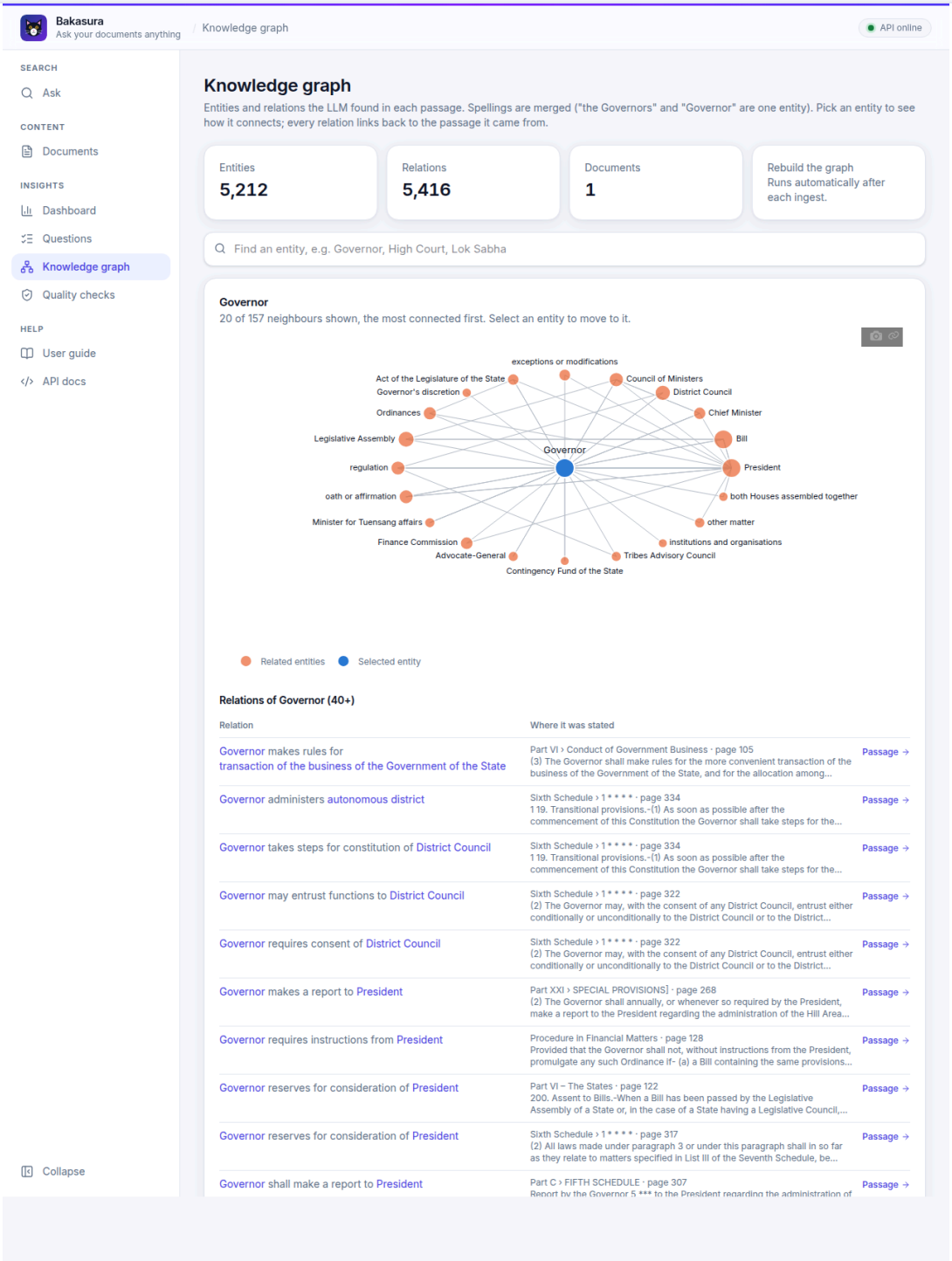
The Questions page listing labelled evaluation questions

This is the answer key for the Dashboard. Each question lists **evidence**: a phrase copied from the passage that answers it. Any section containing the phrase counts as correct, so the labels survive re-chunking.

1. Click **Add**. Give an id, the question in everyday words, and one evidence phrase per line.
2. Choose **test** or **train**. Train questions may be used to fine-tune the model; the Parts of the document holding a test question are kept out of training so the result is not inflated.
3. Under each question the page shows which sections contain its evidence. Not found in any section means a typo or a changed document: an evaluation would leave it out.
4. Click **Save changes**. **Import** and **Export** read and write the JSON file.

The most honest test questions are written by someone who has not read the documents.

Knowledge graph

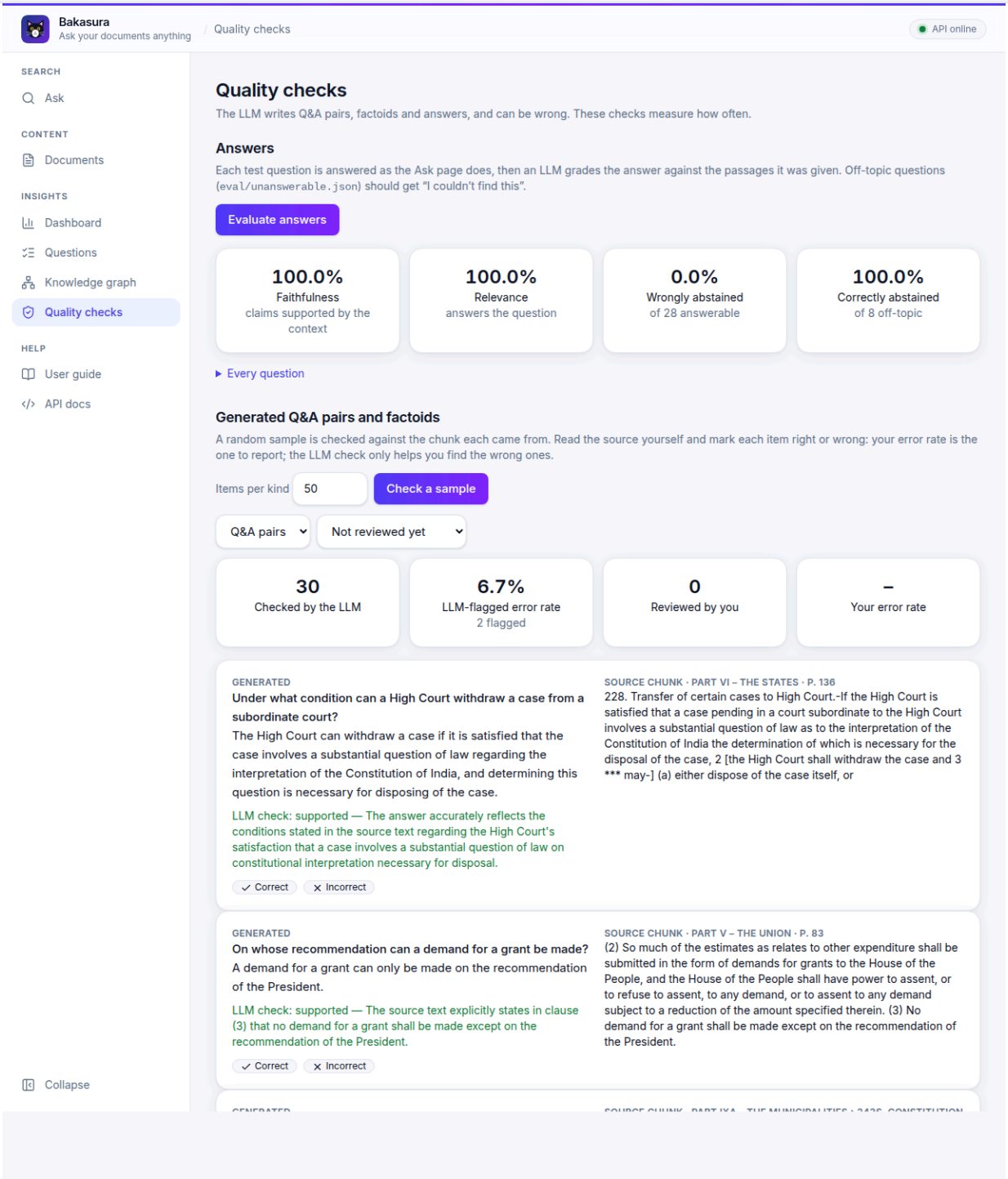


The Knowledge graph page showing an entity, its neighbours and its relations

The graph is built from the relations the LLM found in each chunk (“Governor — appoints — Chief Minister”). Different spellings of one thing merge into one entity, and every relation remembers the chunk that stated it.

- A graph with thousands of entities is unreadable as one picture, so this page shows **one entity at a time**: search for it, or click a bar in the most-connected chart.
- The network shows the entity in the centre and its closest neighbours around it. **Click a neighbour** to move there.
- The table lists the relations with the section, page and a link to the source passage. Always check the source: the relations are only as good as the LLM’s reading.
- **Rebuild** recreates the graph from the stored relations. It also happens automatically after every upload or deletion.

Quality checks



The Quality checks page with answer scores and a generated item next to its source

An LLM wrote the Q&A pairs, factoids and answers, so some are wrong. This page measures how many.

Answers

Click **Evaluate answers**. Every test question is answered as Ask would, and a second LLM call grades each answer against the passages it was given:

- **Faithfulness**: every claim is supported by the passages.

- **Relevance**: the answer addresses the question.
- **Wrongly abstained**: questions that have an answer but the system said “I couldn’t find this”.
- **Correctly abstained**: off-topic questions (listed in `eval/unanswerable.json`) that were refused, as they should be.

Generated Q&A pairs and factoids

1. Choose how many random items to check and click **Check a sample**. Each costs one LLM call.
2. For each item the page shows the generated text beside the chunk it came from, and the LLM’s verdict.
3. Read the source and click **Correct** or **Incorrect** (click again to clear). Use “Flagged by the LLM” to look at the likely errors first.
4. The tiles show the LLM-flagged error rate, your own error rate and how often you agree. **Report your own rate**: it is the trustworthy one, and a sample of 50 to 100 is enough to be useful.

Running and configuring

Start and stop

- `./start.sh` starts Qdrant (Docker), the API and this page. `./stop.sh` stops the API and page; `./stop.sh --all` also stops Qdrant. Your data is kept.
- **Without Docker** (for example on Windows): set `QDRANT_PATH=data/qdrant` in `.env`. The index is then a folder inside the project. To copy an existing index into it, run `uv run python -m capstone.export_index data/qdrant` once. The folder can be zipped and moved to another computer.
- On Windows use `start.ps1` and `stop.ps1`.

Settings that matter

Everything is in `.env` (see `.env.example`). The ones you are most likely to change:

- `LLM_PROVIDER`: which service runs the LLM steps, `groq` (default) or `openai`. This one line decides; the other provider’s lines are ignored.
- `GROQ_API_KEY`, `GROQ_MODEL` or `OPENAI_API_KEY`, `OPENAI_MODEL`: the key (required) and model of the provider you use.
- `LLM_WORKERS`, `LLM_MAX_CONCURRENT`: how many LLM calls run at once. Keep them equal: 4 for Groq, up to 24 for OpenAI.
- `COLLECTION`: the name of the index. **Use a new name** whenever you change the embedding model, then ingest again.

- `DOCUMENT_TTL_DAYS`, `FLUSH_INTERVAL_HOURS`: expiry of new documents.
- `GRAPH_SEARCH`: use the knowledge graph in every search (off by default).
- `ANSWER_MIN_SCORE`: how similar a passage must be to count. Raise it to abstain more often.

Choosing the LLM provider

The LLM writes the Q&A pairs, facts and triplets for every chunk, writes the answers, and checks its own work. You can use either provider. Put the key of the one you want in `.env`, set `LLM_PROVIDER`, then restart (`./stop.sh` and `./start.sh`); the API reads `.env` only when it starts.

- **Groq** (default): fast per call, but it limits how much text it writes per minute, so more than 4 parallel calls only causes retries. Use `LLM_WORKERS=4` and `LLM_MAX_CONCURRENT=4`.
- **OpenAI**: no such cap at the account level we tested, so it speeds up with more parallel calls. Use up to `LLM_WORKERS=24` and `LLM_MAX_CONCURRENT=24`. If you see rate-limit (429) errors, lower them; what your account allows depends on its usage tier.

Measured on 213 chunks of the Constitution, and scaled to the whole document (an estimate, not a promise): OpenAI `gpt-4o-mini` with 24 calls at once, about 7 minutes; Groq `qwen/qwen3.8-27b` at its best, about 17 minutes or more. Speed is not quality: both gave similar counts of Q&A pairs, facts and triplets, but how correct they are is what the Quality checks page measures. OpenAI is a paid service; the whole Constitution costs roughly a dollar with `gpt-4o-mini` (an estimate).

> Switching to a different model means the chunks are enriched again, because the cache is kept per model. Switching back to a model you already used reuses its cached results.

To time both on your own machine, run `uv run python scripts/bench_ingest.py` once with each provider set in `.env`. It enriches the same pages from your PDF in a throwaway index and prints the seconds per chunk. Nothing is written to your real index.
`.env` wins over variables already exported in your terminal. A key left in `~/.bashrc` for another project will not override the one in this project's `.env`.

Troubleshooting

- **"API offline"**: the API is not running or still loading its models (the first start downloads them). Wait a minute, then run `./start.sh` again.
- **The key is missing** (`./start.sh` stops with "Missing GROQ_API_KEY" or "Missing OPENAI_API_KEY"): it checks the key of the provider set in `LLM_PROVIDER` (Groq if the line is absent). Copy `.env.example` to `.env` and put that provider's key in it, or change `LLM_PROVIDER` to the provider whose key you have.

- **“Invalid API Key” (401)**: the key is wrong or was revoked. Create a new one at the provider, put it in `.env` and restart. Nothing is retried for this error, so it shows at once.
- **“model does not exist or you do not have access” (404)**: `GROQ_MODEL` or `OPENAI_MODEL` names a model your key cannot use. Pick another one and restart. This is also shown at once, without retries.
- **Ingestion is slow, or you see rate-limit (429) messages in the API log**: the provider limits how much it will write per minute. Run it again (cached work is not repeated), use fewer parallel calls on Groq (4), or switch to OpenAI, which handles more at once.
- **The Analysis page says “No analysis data yet”**: run `uv run --group eval --group notebook python eval/grid_search.py` (Qdrant running, documents ingested), then reload.
- **The Dashboard has no graph retrievers**: the graph is built after the first ingest. Use Rebuild on the Knowledge graph page.
- **Answers say “I couldn’t find this” for something that is in the documents**: use words closer to the document, or lower `ANSWER_MIN_SCORE` a little.
- **A question is “Not found in any section”** on the Questions page: its evidence phrase is not in the current text. Fix the phrase or remove the question.
- Technical details for developers are on the API docs page.